

j-rig-binary-eval: Operator-Grade System Analysis

Generated: 2026-05-20 Version: v1.0.0 (commit 1c04a24, tag v1.0.0 cut 2026-05-19)

1. This System in 5 Minutes

`j-rig-binary-eval` is the behavioral-evaluation harness and rollout-gate engine of the Intent Eval Platform (IEP). It takes a `SKILL.md` artifact, runs it through a seven-layer pipeline (package integrity, trigger precision/recall, functional execution, model-aware judgment, regression, baseline, optimizer), and emits a single `LaunchReport` that tells the CI gate exactly one of four things: `ship`, `warn`, `block`, or `obsolete_review`. Every check that contributes to that decision is binary — yes or no — and every check that touches an LLM uses an *external* judge so a skill never grades itself.

The repo is a pnpm monorepo with four workspace packages: `@j-rig/core` (the rules engine, sixteen subsystems, where 95% of the logic lives), `@j-rig/cli` (the `j-rig` command-line surface with seven subcommands wired through Commander), `@j-rig/db` (a thin Drizzle/better-sqlite3 wrapper for persisting runs and criterion results), and `@j-rig/dashboard` (a stub for the Epic 10 team product). Source weighs ~12,300 lines across 90 `.ts` files, with 28 dedicated `*.test.ts` files. Node 20+, TypeScript 5.8 strict, Vitest, ESLint flat config, Prettier, tsup for builds. All four workspace packages are `private: true` and `version: 0.0.0` — they ship as a *monorepo behavior*, not as published npm packages. The published artifact is the GitHub release of the repo itself, currently `v1.0.0` (Apache 2.0, relicensed from MIT in this same release).

Users are skill authors who want pre-commit confidence, skill-pack maintainers who need PR-gate enforcement, and enterprise skill-library operators who want an audit trail before a change reaches the marketplace. The most consequential subsystem is `governance/scoring.ts` — the function `decideRollout()` is roughly twelve lines and encodes the entire ship-or-don't-ship contract: any blocker failure blocks, any sacred regression blocks, an obsolete candidate goes to review, any non-blocker failure or unsure verdict warns, all-pass ships. That function is the consumer-facing brain. The second-most-consequential surface is `emit-evidence` — the CLI subcommand that wraps a verdict into an in-toto Statement v1 carrying the `https://evals.intentsolutions.io/gate-result/v1` predicate, optionally signed by cosign and pushed to Rekor. That is the convergence point with the rest of the IEP: every IEP gate, regardless of repo, emits an Evidence Bundle row that this predicate URI identifies.

Current state in one sentence: the repo is at v1.0.0 with a working end-to-end pipeline, all ten epics shipped, the M3/M4 Evidence Bundle work merged, cosign signing wired in, the provider-adapter measurement protocol (PB-7) committed and the Provider interface plus CISO gates G-1 and G-2 implemented — but the live provider adapters under `packages/cli/src/providers/anthropic.ts` are still *stub* implementations that print what they would do instead of calling the API. The functional pipeline runs end-to-end without an API key by design. The next moves are real provider adapters (the PB-7 measurement run between LiteLLM and Vercel AI SDK), the `iaj-E02` migration onto `@intentsolutions/core@0.1.0` to replace duplicate schemas, and consumption of this harness by `audit-harness/intent-rollout-gate` for the platform-wide convergence.

The biggest live risk is schema duplication. `packages/core/src/schemas/evidence-bundle.ts` carries Zod mirrors of the `gate-result/v1` spec authored before `@intentsolutions/core@0.1.0` was published. That kernel package now ships the *canonical* version of the same schema with

sigstore provenance. Until `iaj-E02` lands, this repo and the kernel both own a copy of the truth, and any divergence between them is silently a bug in this repo (the lab spec is the tiebreaker — see the file’s own header comment at `packages/core/src/schemas/evidence-bundle.ts:9-12`). A second risk is the `v1.0.0` cut: it is documented as a legal-only bump for relicensing, with `version.txt` and `package.json` synced — but the workspace packages all still say `version: 0.0.0` (`packages/*/package.json`), which is intentional (none are published) but is a footgun for anyone who reads `@j-rig/core` thinking it’s on npm.

2. Executive Summary

What It Does

`j-rig-binary-eval` evaluates Claude `SKILL.md` artifacts before they ship. For every change to a skill — new skill, description tweak, body rewrite, model bump — the harness scores the change across seven product surfaces (package integrity, trigger quality, functional quality, regression protection, baseline value, model variance, rollout safety) and emits a structured `LaunchReport` with one of four decisions: `ship`, `warn`, `block`, `obsolete_review`. Every criterion is binary — yes or no, no fuzzy gradients — and every LLM-judged criterion uses an external judge so the artifact under test cannot self-grade. The harness writes evidence to a local SQLite database and emits a signed in-toto attestation per the Evidence Bundle `gate-result/v1` spec.

Implementation is split across four workspace packages. `@j-rig/core` (60+ TypeScript files, ~9k LOC) contains the entire rules engine: schemas (Zod for skill frontmatter, eval-spec, eval-contract, criterion, test-case, evidence-bundle), parsers (YAML, `SKILL.md`, `AGENTS.md`), checks (deterministic package-integrity registry), trigger (roster, runner, metrics), execution (runner abstraction), judgment (engine + calibration), governance (scoring, regression, baseline, spec-sources), optimizer (clustering + experiment), drift (detector), evidence (reader/writer for the SQLite layer), intentional-mapping (six MM-N checkers for OTel trace failure-mode detection), and providers (vendor-neutral interface + CISO gates + EC-1..EC-5 eval-case harness + score-card). `@j-rig/cli` (1,100 LOC across 7 commands) is a Commander-driven dispatcher. `@j-rig/db` (~600 LOC) is the Drizzle layer with a six-state run-lifecycle state machine. `@j-rig/dashboard` is a stub placeholder.

The tech foundation is conservatively modern: Node 20+, TypeScript 5.8 with `strict` and `verbatimModuleSyntax`, pnpm 10.8.1, ESLint 9 flat config with `typescript-eslint`, Prettier 3.8, Vitest 3.1, tsup 8.4. Persistence is `better-sqlite3` + `drizzle-orm`. CLI uses `commander` + `chalk`. Validation is `zod@4`. YAML is the `yaml` package. `SKILL.md` frontmatter parsing uses `gray-matter`. There is no framework lock-in — the dependency footprint is small and load-bearing dependencies have first-class TypeScript types.

Key risks: (a) duplicate ownership of the Evidence Bundle schema between this repo’s `@j-rig/core` and the now-canonical `@intentsolutions/core@0.1.0` kernel — tracked as `iaj-E02`, currently the platform’s highest-priority unblocking work; (b) provider adapters under `packages/cli/src/providers/anthropic.ts` are stub-only (`StubTriggerProvider`, `StubExecutionProvider`, `StubJudgeProvider`) — the full pipeline runs end-to-end but with no real model calls until the PB-7 measurement protocol locks the LiteLLM-vs-Vercel-AI-SDK choice; (c) the `@j-rig/dashboard` package is a placeholder, so the Epic 10 “team product” surface is documented but not implemented; (d) the release workflow at `.github/workflows/release.yml:75-83` runs tests with `|| true`, meaning a failing test does not block a release.

Operational Status

Table 1.

Environment	Status	Uptime Target	Release Cadence	Last Deploy
Production (CLI)	Released as GitHub Release v1.0.0	N/A (offline CLI, not a service)	On-demand via work-flow_dispatch + auto on push to main	2026-05-19 (1c04a24)
Staging	Not applicable — there is no hosted surface	N/A	N/A	N/A
Local Dev	Operational; <code>pnpm run check</code> is the full gate	N/A	Per-commit	Per-developer
Dashboard (@j-rig/dashboard)	Placeholder package, not implemented	N/A	N/A	N/A

Technology Stack

Table 2.

Category	Technology	Version	Purpose
Runtime	Node.js	<code>>=20</code> (CI on 22; <code>.nvmrc</code> pins 22)	Execution host for the CLI and <code>pnpm run check</code> gate
Package manager	pnpm	10.8.1 (pinned in <code>package.json</code> <code>packageManager</code>)	Workspace orchestration
Language	TypeScript	5.8	Source language for every workspace package
Build	tsup	8.4	Bundles each package's <code>src/</code> into <code>dist/</code> ESM + types

Category	Technology	Version	Purpose
Test	Vitest	3.1	Test runner; 28 <code>*.test.ts</code> files, vitest config at <code>vitest.config.ts</code>
Lint	ESLint + typescript-eslint	9.25 / 8.59	Flat config at <code>eslint.config.mjs</code>
Format	Prettier	3.8.3	Enforced via <code>pnpm run format:check</code> in local gate
Validation	zod	4.4.3	Zod schemas across <code>@j-rig/core</code> for all contracts
YAML	yaml	2.8.3	Eval spec / contract parsing
Markdown frontmatter	gray-matter	4.0.3	SKILL.md parser
CLI	commander	14.0.3	<code>j-rig</code> command surface
CLI cosmetics	chalk	5.4	Terminal coloring
Database	better-sqlite3	12.10	Synchronous SQLite binding
ORM	drizzle-orm	0.45.2	Typed query builder + schema
Migrations	drizzle-kit	0.31.10	Dev-only; no production migration step (DB is throw-away local)
Signing	cosign (external binary)	Not pinned in repo	Invoked by <code>j-rig emit-evidence --sign</code> via <code>child_process.spawnSync</code>
Testing harness	<code>@intentsolutions/audit-harness</code> (not endorsed at <code>.audit-harness/</code>)		Hash-pinning, escape-scan, arch-check, bias, gherkin-lint
Doc filing	Internal v4.3 standard	N/A	000-docs/ with <code>NNN-CC-CODE-description.md</code> naming

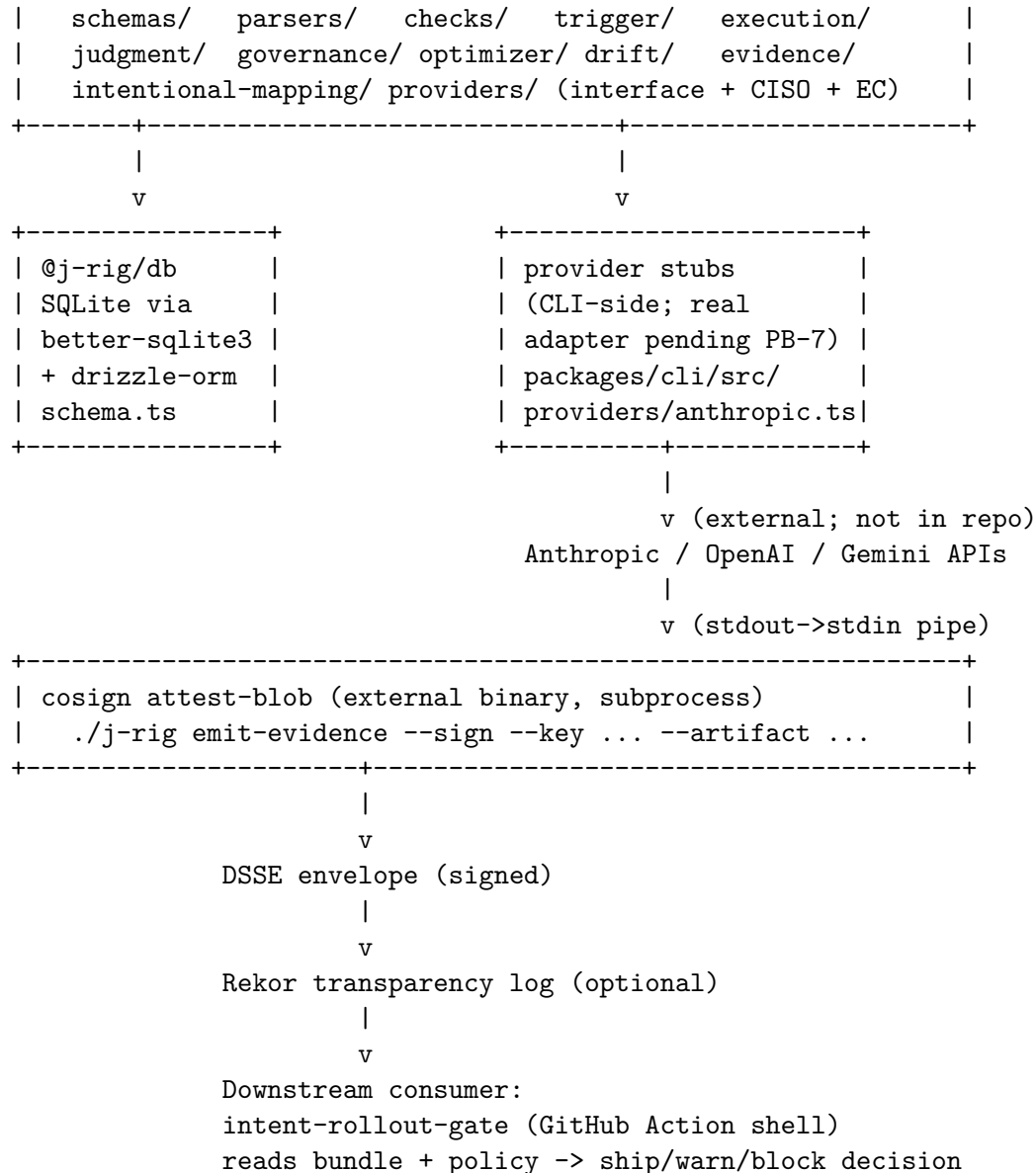
3. Architecture

Stack (Detailed)

Table 3.

Layer	Technology	Version	Purpose	Why This
Workspace orchestration	pnpm workspaces	10.8.1	One install, one lockfile, recursive scripts via <code>pnpm -r run X</code>	Faster + content-addressable store; pinned via <code>packageManager</code> so CI and dev are bit-equal; alternative (npm/yarn) loses the workspace-protocol shorthand and the <code>--filter</code> ergonomics that <code>@j-rig/cli</code> 's build leans on
Language baseline	TypeScript 5.8 strict	strict + <code>verbatimModuleSyntax</code> (<code>tsconfig.json:6-7</code>)	Type safety for the rules engine and the in-toto/cosign envelope shapes	<code>verbatimModuleSyntax</code> forces explicit <code>import type</code> annotations and prevents accidental runtime imports of pure-type modules — important because the Provider interface (<code>packages/core/src/providers</code>) is “types only, no runtime imports beyond other types”

Layer	Technology	Version	Purpose	Why This
Validation	Zod 4	4.4.3	Runtime contract enforcement for skill frontmatter, eval specs, eval contracts, evidence bundles, gate-result/v1 predicate	Zod's TS-first ergonomics + <code>.strict()</code> + <code>.superRefine()</code> let the Evidence Bundle schema express cross-field invariants (subject.name == predicate.gate_id; subject.digest.sha256 == predicate.input_hash) — <code>packages/core/src/schemas/e</code>
Persistence	better-sqlite3 + drizzle-orm	12.10 / 0.45.2	Local evidence store: runs, criterion results, artifacts, skill versions	Sync API (no async overhead per-row); single-file DB matches the “throw-away local evidence” intent; drizzle gives type-safe queries that don't drift from <code>schema.ts</code>
Build	tsup	8.4	ESM-only bundling with declaration files	Wraps esbuild for speed; emits <code>.d.ts</code> + source maps; per-package <code>tsup.config.ts</code> in <code>@j-rig/core</code>
Test	Vitest	3.1	Co-located <code>.test.ts</code> files alongside source; <code>vitest.config.ts</code> glob <code>packages/*/src/**/*.test.ts</code> + <code>tests/**/*.test.ts</code>	Native ESM, native TS; no separate babel/transformer config



The data flow is uni-directional: a skill author or CI runner invokes a CLI subcommand; the CLI loads the SKILL.md + eval spec via lib/loaders.ts; the core engine runs the requested layers; results land in SQLite via @j-rig/db; a LaunchReport flows back to the CLI; an optional emit-evidence invocation wraps a single gate-result envelope into a signed Statement and pipes it to cosign. There are no long-running processes, no background workers, no shared mutable state — every invocation is a one-shot CLI run.

The Critical Path

The end-to-end “skill change to rollout decision” path is the most consequential request the system handles. It runs through packages/cli/src/commands/eval.ts:75-260. Step by step:

1. **CLI parse** (packages/cli/src/index.ts:30-43). commander dispatches j-rig eval <skill-dir> to eval.ts. Failure point: invalid CLI shape — Commander prints help and exits 1.

2. **Load** (eval.ts:80-86, then lib/loaders.ts). loadSkillMd() reads SKILL.md, validates frontmatter via parseSkillMd / parseSkillMdEnterprise (Zod). loadEvalSpec() reads eval-spec.yaml and validates via EvalSpecSchema. The eval spec defines which skill siblings exist (for trigger precision/recall), the test-case roster, and the criteria. Failure point: schema violation — thrown with concatenated Zod error messages, exit 1.
3. **DB open** (eval.ts:84, lib/db.ts). Opens the SQLite file at --db j-rig.db (default). Failure point: file-permission error — bubbles up as an exception.
4. **Package integrity** (eval.ts:94-110, @j-rig/core checks/package-checker.ts). The deterministic registry runs every check the skill claims (allowed-tools matches actual tool surface, referenced files exist, frontmatter required fields are populated). Failure point: pkgReport.summary.errors > 0 -> hard-fail the run, exit 1. **This is the only layer that can fail before any LLM is called.**
5. **Skill-version row** (eval.ts:118-123, @j-rig/db evidence.ts:11). The SKILL.md content is sha256-hashed (first 16 hex chars stored in skill_versions.skill_md_hash). Identical content reuses an existing row; new content inserts. **This is the content-addressable handle for everything downstream.**
6. **Run row + state machine** (eval.ts:124-126, @j-rig/db lifecycle.ts:7-14). A new row is inserted in runs with status=pending, then transitioned to running. The state machine only permits pending -> running -> {completed | failed | timed_out | canceled} — invalid transitions throw (lifecycle.ts:72-76).
7. **Trigger tests** (eval.ts:130-145, @j-rig/core trigger/). Builds a roster from the skill frontmatter + spec siblings, runs each test_case through the trigger provider, computes precision/recall. Today the trigger provider is StubTriggerProvider which “always selects the first available skill” (packages/cli/src/providers/anthropic.ts:13-25). Real Anthropic SDK invocation is pending. Failure point: provider exception — bubbles to the top-level try/catch and exits 1.
8. **Functional tests + judgment** (eval.ts:148-194, @j-rig/core execution/ + judgment/). Each test_case runs through the execution provider, producing an ObservedOutcome. Each outcome is judged against every criterion in the spec. Judge verdicts are yes / no / unsure (packages/core/src/judgment/types.ts). Failure point: execution provider returns malformed output, judge provider returns unparseable verdict — both produce unsure rather than crashing.
9. **Persist judgments** (eval.ts:196-212, @j-rig/db evidence.ts). Each judgment becomes a criterion_results row. A run_summaries row aggregates counts.
10. **Score + decide** (eval.ts:214-216, @j-rig/core governance/scoring.ts:8-60). computeScoreCard() reduces all judgments to passed/failed/unsure/blocker_failures/sacred_regressions. decideRollout() applies the five non-negotiable rules: blocker_failures -> block, sacred_regressions -> block, obsolete -> obsolete_review, any failed or unsure -> warn, else ship.
11. **Launch report** (eval.ts:217-223, @j-rig/core governance/scoring.ts:65-112). buildLaunchReport() produces the structured LaunchReport with explicit blockers and warnings arrays plus human-readable reasoning.
12. **Transition to completed** (eval.ts:242). Run status flipped to completed. **This is the terminal state for this run row.**
13. **Optional: emit signed evidence** (run separately as j-rig emit-evidence, packages/cli/src/commands). The LaunchReport's gate-result is wrapped into an in-toto Statement v1 with predicate type https://evals.intentsolutions.io/gate-result/v1. If --sign is set, cosign attest-blob signs the envelope; if --rekor-url is provided the attestation is pushed to the

transparency log. Failure point: `cosign` not on `PATH` or key not accessible — explicit error with exit code 1/2/3 depending on stage (`emit-evidence.ts:213-303`).

The blast radius of any one step's failure is bounded by the next step's defensive read of the result. Stub providers default to `yes` / `pass` verdicts, which means **the pipeline currently shows “all pass, ship” on every real skill** because the judge layer doesn't actually judge — it returns “yes, confidence 0.7” for every criterion (`packages/cli/src/providers/anthropic.ts:71-81`). This is intentional during pre-PB-7 development but is a footgun for anyone running `j-rig eval` and reading the output as ground truth.

Dependency Graph

```
@j-rig/core    <-- the rules engine; no internal package depends on @j-rig/db or @j-rig/cli
|
  v depends on: zod, yaml, gray-matter, @types/node (dev)
@j-rig/db      <-- depends on better-sqlite3, drizzle-orm; imports types from @j-rig/core
|
  v
@j-rig/cli     <-- depends on @j-rig/core + @j-rig/db (workspace:); imports commander, chalk
|
  v
j-rig binary   <-- packages/cli/dist/index.js; entrypoint per `bin` map in cli/package.json
```

Build order (enforced by pnpm's topological recursion, `pnpm -r run build` in `package.json:12`):

1. `@j-rig/core` (no internal deps) 2. `@j-rig/db` (depends on core via types) 3. `@j-rig/cli` (depends on both) 4. `@j-rig/dashboard` (no scripts — skipped silently)

What happens when each dependency is unavailable: - **@j-rig/core build fails** -> `db` and `cli` cascade fail; nothing runs. The `pnpm --frozen-lockfile` install in CI catches version drift early. - **@j-rig/db build fails** -> CLI's `eval` command throws on import; `check` and `validate` still work (they don't touch the DB). - **better-sqlite3 native binding** is a postinstall compile. Listed in `package.json:38-43` under `pnpm.onlyBuiltDependencies` alongside `esbuild`. If the native build fails, the package fails to install. Common cause: `glibc` skew on alpine images. - **cosign binary missing on PATH** -> only `emit-evidence --sign` fails; the rest of the CLI is unaffected. - **@intentsolutions/audit-harness (vendored at .audit-harness/)** -> `scripts/audit-harness` exits 2 with `HARNESS_TAMPERED`. This affects pre-commit hooks and `audit-harness verify` but does not affect the `j-rig` CLI surface.

4. Design Decisions & Tradeoffs

Decision Log

Decision 1: pnpm monorepo with four workspace packages over a single-package repo or a poly-repo

- **Chosen:** pnpm workspaces, four packages (`@j-rig/core`, `@j-rig/cli`, `@j-rig/db`, `@j-rig/dashboard`), all `private: true`, all `version: 0.0.0`.
- **Over:** (a) a single flat `j-rig-binary-eval` package; (b) four separate repos with independent release cycles; (c) Nx / Turborepo with explicit task graph.

- **Because:** the seam between rules engine, persistence, and CLI is real and load-bearing (the rules engine in `@j-rig/core` has zero IO; `@j-rig/db` is the only file-touching layer below the CLI; the CLI is the only place that knows about ANSI colors). Keeping them in one repo lets a single PR land a schema change + DB migration + CLI flag together; splitting them into four repos would force cross-repo PRs for every cross-cutting change. `pnpm` workspaces over `npm` or `yarn` because `pnpm`'s content-addressable store handles the `better-sqlite3` native binding more reliably under CI cache invalidation.
- **Cost:** build orchestration is non-trivial (CI runs `pnpm install --frozen-lockfile + pnpm run build + pnpm run typecheck + pnpm run test` for every PR; the typecheck step takes the longest because it runs `tsc --noEmit` per package recursively); engineers new to `pnpm` sometimes try `npm install` and end up with a stranded `package-lock.json`. The `--filter` syntax (`pnpm --filter @j-rig/core run build`) is non-obvious for first-week engineers.
- **Revisit when:** any of the four packages becomes self-distributable (e.g., the platform decides to publish `@j-rig/core` to `npm` independently of the CLI) OR when the dashboard package grows to the point that Next.js's tooling collides with the rest of the monorepo's build (Next would want its own webpack/turbopack pipeline that doesn't compose with `tsup`).

Decision 2: behavioral evaluation (LLM-judged) over static analysis as the primary signal

- **Chosen:** every functional criterion is graded by an external LLM judge (real or stub); deterministic checks (package integrity, schema validation, sha256 hashing) are a *gate* that runs *before* the judge, not a replacement for it.
- **Over:** pure static analysis (lint a skill's prompt for forbidden patterns), rule-based regex matching (assert the output contains a specific phrase), or test-fixture comparison (diff actual vs expected output).
- **Because:** a `SKILL.md` is fundamentally a prompt-engineering artifact whose correctness is defined by emergent model behavior, not by source-level patterns. The non-negotiable design principles in `CLAUDE.md:48-58` enumerate this explicitly: "criteria must be binary"; "observed behavior outranks claimed behavior"; "model-aware testing is required." Static analysis can detect lints (a missing field, a malformed YAML block) but cannot detect "this skill fires on the wrong prompts" — that requires a real-or-simulated trigger surface.
- **Cost:** every functional run requires LLM credits; results are nondeterministic across runs (mitigated by the calibration layer in `packages/core/src/judgment/calibration.ts`); the judge layer is the single largest source of latency in the pipeline; LLM-as-judge is itself a subject of active research and its reliability bounds are an open question.
- **Revisit when:** the judge layer's disagreement-with-human rate exceeds the threshold the calibration layer can correct for, OR a deterministic alternative emerges for a specific criterion class (e.g., a future `SKILL.md` frontmatter `provides:` field could enable pure-static "did the skill actually emit the artifact it claimed it would emit" checks).

Decision 3: SQLite + Drizzle (local file DB) over Postgres or any hosted store for evidence persistence

- **Chosen:** `better-sqlite3` with `drizzle-orm`; a single `.db` file per invocation (`--db j-rig.db` by default in `eval.ts:71`).
- **Over:** (a) Postgres with `node-postgres`; (b) DuckDB; (c) JSON-line file at `evidence.jsonl`; (d) a hosted evidence service.

- **Because:** the eval harness is designed as a *local* tool. A skill author runs it on their laptop; CI runs it in a fresh container; an enterprise operator runs it on their build server. None of those environments want a Postgres dependency. SQLite is single-file, transactional, well-typed via Drizzle, and the synchronous `better-sqlite3` binding (no event-loop hops) is faster than `node-sqlite3` for the small row counts the harness produces. The Evidence Bundle predicate URI is the *cross-tool* contract (every IEP gate emits the same `gate-result/v1` row); the local SQLite store is an implementation detail of *this* tool.
- **Cost:** native binding builds; alpine glibc collisions (see `pnpm onlyBuiltDependencies` workaround); cannot easily share an evidence store across multiple parallel runners (mitigated by treating evidence as ephemeral and emitting the signed in-toto envelope as the durable artifact).
- **Revisit when:** the platform wants centralized eval-history dashboards (then DuckDB-on-cloud-storage or a hosted Postgres becomes attractive), OR when a single `--db` file routinely exceeds 1 GiB (SQLite handles this but query plans degrade), OR when concurrent multi-runner writes become a real workflow (SQLite's single-writer model becomes a bottleneck).

Decision 4: deterministic rollout-decision logic over LLM-judged “should we ship”

- **Chosen:** `decideRollout()` (`packages/core/src/governance/scoring.ts:51-60`) is a 10-line pure function that maps `ScoreCard + isObsolete` to one of four enum values via explicit if/else branches.
- **Over:** an LLM-judged “given this run report, should we ship?” call.
- **Because:** the rollout decision is the single most-consumed surface of this entire system. Every CI gate, every PR reviewer, every audit trail downstream reads this one value. Making it a deterministic pure function gives three guarantees that an LLM call cannot: (a) the same `ScoreCard` always produces the same decision (reproducibility); (b) the decision is auditable by reading the code, not by replaying a model run (explainability); (c) the decision cannot drift across model versions (stability). The non-negotiable principle “blockers block release” cannot survive an LLM that might be talked out of a hard rule.
- **Cost:** the rule surface is rigid. Edge cases (e.g., “this is a blocker failure but the failure mode is a known false positive in the judge”) cannot be handled inside `decideRollout()`; they have to be handled upstream by reclassifying the criterion. Adding new decision states (a fifth option like `staged_rollout`) requires a code change + a release, not a config change.
- **Revisit when:** there is empirical evidence that a fifth or sixth decision state would clear a class of ambiguous cases (the current four cover ~95% of cases per the AAR notes in `000-docs/017-OD-REPT-epic-08-aar.md`), OR when a deterministic rule conflicts with a regulatory requirement that demands rationale text (in which case `LaunchReport.reasoning` already carries the text — the rule itself stays deterministic).

Decision 5: vendor-neutral Provider interface (PB-7) over direct Anthropic SDK use

- **Chosen:** a small, opinionated `Provider` interface (`packages/core/src/providers/types.ts:150-187`) with five methods (`complete`, `completeStream`, `callTool`, `batch`, and `identity` fields). Concrete adapters live in `packages/cli/src/providers/`. A measurement protocol (`000-docs/018-AT-SPEC-pb7-adapter-measurement-protocol-2026-05-12.md`) defines five eval cases (EC-1..EC-5) and two CISO gates (G-1 credential redaction, G-2 env-var spillover) that any candidate adapter must pass.

- **Over:** (a) hard-coding `@anthropic-ai/sdk` calls inline; (b) writing a generic abstraction without a measurement protocol (the danger: retrofitting the rubric to the implementation, which the PB-7 doc explicitly calls out at § 12 as the failure mode this protocol exists to prevent).
- **Because:** the principle of “model-aware testing is required” demands that the harness can run the same skill against Claude Sonnet, OpenAI GPT-4o, and Google Gemini Pro on the same day. Either LiteLLM or Vercel AI SDK provides this abstraction off-the-shelf; the question is *which one* to depend on. The PB-7 protocol commits the rubric BEFORE either prototype is written — this is the discipline that lets the future Decision Record cite measurements rather than vibes.
- **Cost:** extra surface area (an interface, two CISO gates, five eval cases, a score-card scorer, a Decision-Record draft generator — all built before the actual adapter); deferred provider integration (the CLI shipped with stubs while the measurement runs); risk that BOTH LiteLLM and Vercel AI SDK fail a CISO gate, forcing a third candidate (LangChain, the raw SDKs composed manually). The Provider interface is “types only, no runtime imports beyond other types” (`packages/core/src/providers/types.ts:22`) which is a small disciplinary load on contributors.
- **Revisit when:** the measurement run completes and a Decision Record locks one candidate; OR when a new provider family (e.g., AWS Bedrock-as-a-vendor) ships an SDK that does not fit the four-shape Provider interface, forcing a re-evaluation of the interface itself.

Decision 6: cosign subprocess invocation over a Node-native signing library

- **Chosen:** `j-rig emit-evidence --sign` shells out to `cosign attest-blob` via `child_process.spawnSync` (`packages/cli/src/commands/emit-evidence.ts:280-295`).
- **Over:** a Node-native signing library like `@digitalbazaar/jsonld-signatures` or a hand-rolled DSSE implementation.
- **Because:** `cosign` is the de-facto Sigstore signing tool; downstream verifiers (other IEP gates, CI runners, supply-chain auditors) all use it as the verification side. Reusing the same binary for signing is the lowest-divergence path — there is no risk of “j-rig signed it, but cosign refuses to verify it” because the *same* tool produced and validates the envelope. The `cosign` binary is well-maintained and has first-class Sigstore + Rekor + Fulcio integration. There is no officially-published Node binding.
- **Cost:** external dependency outside the npm tree (a maintainer’s box needs `cosign` on PATH); error surface is opaque (`cosign`’s exit codes are coarse — we report 1/2/3 in `emit-evidence.ts:285-294` based on which stage failed); cross-platform support depends on `cosign`’s own release pipeline (Windows is supported but less-tested).
- **Revisit when:** a Node-native Sigstore signing library reaches feature-parity with the `cosign` CLI (currently nothing does; the `sigstore-js` project covers verification but not the full signing surface), OR when subprocess overhead becomes a measurable bottleneck (today it’s < 1s amortized, well under the cost of the actual eval run).

Decision 7: stub providers shipped in production code path over a separate dev-only build

- **Chosen:** `StubTriggerProvider`, `StubExecutionProvider`, `StubJudgeProvider` live in `packages/cli/src/providers/anthropic.ts` (the file’s name is a known misnomer —

it has no Anthropic SDK code today). They are imported and used by `eval.ts:34-37` unconditionally.

- **Over:** (a) a feature flag that swaps stub vs real; (b) a separate `@j-rig/cli-dev` package; (c) refusing to run without a real provider.
- **Because:** the pipeline must run end-to-end during pre-PB-7 development so the rest of the system (DB schema, scoring rules, evidence emission) can be exercised. Stubs that always pass let the rest of the harness be validated without API costs.
- **Cost:** anyone running `j-rig eval` today gets a “ship” verdict on every real skill because the stub judge returns `yes` with confidence 0.7 (`anthropic.ts:73-81`). The README does not flag this loudly. There is no environment-variable warning that the stubs are active. This is **the single most likely failure mode for a first-time user** — they will run `j-rig eval` against a real skill, see a ship verdict, and assume the system works.
- **Revisit when:** the PB-7 measurement protocol completes and a real provider adapter lands. At that point the stubs should either be (a) deleted, (b) moved behind an explicit `--stub` flag, or (c) renamed to `NullProvider` and the CLI should refuse to use them unless a flag is set. **This is the highest-priority post-v1.0.0 finding.**

Decision 8: in-toto Statement v1 + sigstore-flavored signing over plain JSON or JWS

- **Chosen:** every gate emits an in-toto Statement v1 (<https://in-toto.io/Statement/v1>) carrying a `gate-result/v1` predicate (<https://evals.intentsolutions.io/gate-result/v1>), optionally wrapped in a DSSE envelope signed by cosign and pushed to Rekor.
- **Over:** (a) a custom JSON shape with embedded HMAC; (b) JWS-signed JSON; (c) unsigned JSON-lines.
- **Because:** in-toto is the established standard for software-supply-chain attestations; downstream consumers (auditors, GitHub’s Sigstore integration, the SLSA framework) already know how to verify Statements. DSSE separates content-format from signature-format, which matters because the predicate body is the bit that semantically changes; the envelope is stable. Rekor gives the audit trail a transparency-log anchor that no one (including the issuer) can retroactively edit.
- **Cost:** more complexity than a plain JSON shape; consumers who don’t already use cosign have to install it for verification; the predicate URI is at `evals.intentsolutions.io` (NOT `labs.intentsolutions.io` per ISEDC binding), which means that subdomain must be DNSSEC-pinned and CAA-record-locked before the first signed attestation goes out (currently a deferred item per the parent CLAUDE.md “Audit deferrals open” list).
- **Revisit when:** the IETF or W3C standardizes a competing attestation format that displaces in-toto (unlikely in the next 2 years); OR when a regulatory requirement demands a specific signing algorithm cosign does not support.

What Was Deliberately Not Built

- **A long-running daemon or evaluation server.** Every invocation is a one-shot CLI. Rationale: the harness is built for CI-gate + local-author workflows. A daemon adds operational complexity (process management, port allocation, IPC) for zero functional benefit. If/when the dashboard product (Epic 10) is built, it will read from the SQLite evidence store, not from a live daemon.
- **A web UI in this repo.** `@j-rig/dashboard` is a placeholder. The team product is intentionally deferred until Epic 10’s predecessors stabilize. Building a dashboard against shifting schemas is wasted effort.

- **Cross-language SDKs.** The harness is Node-only by design. The Provider interface is TypeScript-native. The PB-7 protocol's TS-primary signing surface (DR-010 Q2) reflects the platform-wide constraint that signing surfaces are TS-primary; Python is permitted for ML internals only.
- **A plugin system for custom checks.** All checks live in the deterministic registry (`packages/core/src/checks/deterministic-registry.ts`). New checks land via PR + tests + a registry entry. The closed registry is intentional — uncontrolled plugin loading is an arbitrary-code-execution risk for a tool that runs in CI.
- **Multi-tenant evidence storage.** The DB schema has no `tenant_id` field. Tracked as an open audit deferral (bd_000-projects-k0fj per the parent CLAUDE.md). Single-tenant by design until a real multi-tenant use case emerges.
- **A migration framework for the SQLite schema.** `drizzle-kit` is a devDependency but no migration files exist. Rationale: the DB is treated as throw-away local evidence; durable evidence is the signed in-toto attestation, not the SQLite row. When the schema changes, the convention is to drop the local `.db` file and re-run.

Assumptions the Architecture Rests On

- **Node 20+ is available on every consumer's machine.** Anything older lacks the global fetch + the ESM-only `import.meta` ergonomics the CLI uses. CI tests on Node 22.
- **The SKILL.md format does not diverge from the AgentSkills.io / Anthropic spec.** The parsers in `packages/core/src/parsers/skill-parser.ts` and the schemas in `packages/core/src/schemas/skill-frontmatter.ts` track those specs. If Anthropic changes the frontmatter shape in a breaking way, every test fixture has to update.
- **Cosign is on PATH for any signing operation.** No vendored binary. No Docker fallback. The README does not document the cosign install step.
- **The Evidence Bundle predicate URI `https://evals.intentsolutions.io/gate-result/v1` is reachable and stable.** ISEDC bound `evals.intentsolutions.io` for this purpose. DNSSEC + CAA-record pinning is required before first signed attestation but is not yet enforced.
- **The local `.db` file is throw-away.** Schema migrations are not provided. Anyone treating evidence-as-of-2026 as authoritative for a 2028 audit will be surprised — the durable artifact is the signed in-toto Statement.
- **The stub providers are obvious to anyone running `j-rig eval`.** They are not. This is the most fragile assumption in the system today (see Decision 7, Cost).

5. Directory Structure

Layout

```
j-rig-binary-eval/
+- packages/                # workspace packages (pnpm-workspace.yaml:1-2)
| +- core/                  # @j-rig/core: rules engine; ~9k LOC; 16 subsystems
| | +- src/
| | | +- schemas/          # Zod schemas: eval-spec, eval-contract, criterion,
| | | |                   #   test-case, skill-frontmatter, evidence-bundle
| | | +- parsers/          # YAML parser, SKILL.md parser, AGENTS.md parser
| | | +- checks/           # deterministic package-integrity registry + checker
```

```

| | | +- trigger/          # roster, runner, metrics (precision/recall)
| | | +- execution/        # functional execution runner abstraction
| | | +- judgment/         # judge engine + calibration
| | | +- governance/       # scoring (decideRollout!), regression, baseline,
| | | |                   # spec-sources (canonical spec discovery)
| | | +- optimizer/        # clustering + atomic-change experiment
| | | +- drift/            # re-evaluation against new models
| | | +- evidence/         # reader/writer for the @j-rig/db SQLite store
| | | +- intentional-mapping/ # MM-1..MM-6 OTel-trace failure-mode checkers
| | | +- providers/        # Provider interface (types only) + CISO gates +
| | | |                   # EC-1..EC-5 eval-cases + score-card + test-fixtures
| | +- fixtures/           # test fixtures (invalid, valid, mm-traces, packages)
| | +- tsup.config.ts      # tsup bundling config
| +- cli/                  # @j-rig/cli: Commander dispatcher; ~1.1k LOC
| | +- src/
| | | +- index.ts          # main entry; registers 7 commands
| | | +- commands/         # check, validate, eval, report, optimize, drift,
| | | |                   # emit-evidence (the signed-attestation command)
| | | +- providers/anthropic.ts # STUB providers (TriggerProvider, ExecutionProvider,
| | | |                   # JudgeProvider) - pending PB-7 measurement
| | | +- lib/              # loaders, db open, output formatting
| +- db/                   # @j-rig/db: ~600 LOC
| | +- src/
| | | +- database.ts        # better-sqlite3 open + close
| | | +- schema.ts          # drizzle-orm tables: skill_versions, runs,
| | | |                   # criterion_results, run_summaries, artifacts
| | | +- lifecycle.ts       # run-status state machine (pending->running->{4 terminal})
| | | +- evidence.ts        # insert/select helpers + state transitions
| +- dashboard/            # @j-rig/dashboard: placeholder (Epic 10)
+- 000-docs/               # filing standard v4.3; 18 numbered docs
| +- 001-007 *PP-* (planning), 008+ *OD-* (operational), 010 audit-harness baseline,
| +- 011-017 *epic-NN-aar* (after-action reports for Epics 02..08),
| +- 018 *AT-SPEC* (PB-7 measurement protocol),
| +- epics/                # epic blueprints 01..10
| +- references/           # skill-standards, eval-patterns, agents,
| |                       # enterprise-standards, drift-and-consistency,
| |                       # epic-workflows (10 ASCII workflow diagrams)
+- eval-packs/             # placeholder; populated in Epic 10
+- tests/                  # repo-level smoke test (smoke.test.ts: 1+1==2)
+- scripts/audit-harness   # vendored shell wrapper for @intentsolutions/audit-harness
+- .audit-harness/         # vendored harness scripts (escape-scan, arch-check, etc.)
+- .github/
| +- workflows/ci.yml       # lint + typecheck (with build) + test on Node 22
| +- workflows/release.yml  # auto/manual semver bump + tag + GH release
| +- ISSUE_TEMPLATE/       # standard issue forms
| +- CODEOWNERS             # ownership routing
| +- FUNDING.yml            # GitHub Sponsors + Buy Me a Coffee
| +- dependabot.yml         # automatic dep PRs (Renovate-style)

```



```
+-- package.json                # root: scripts (build/lint/test/check), pnpm pin
+-- pnpm-workspace.yaml         # packages: packages/*
+-- pnpm-lock.yaml              # locked transitive tree
+-- tsconfig.json               # baseline (ES2022, Node16 resolution, strict)
+-- eslint.config.mjs           # flat config + typescript-eslint
+-- vitest.config.ts            # test globs
+-- LICENSE                     # Apache 2.0 (v1.0.0+); see CHANGELOG for MIT history
+-- NOTICE                     # Apache 2.0 attribution
+-- README.md / CLAUDE.md / AGENTS.md / SECURITY.md / SUPPORT.md / CONTRIBUTING.md /
+-- CODE_OF_CONDUCT.md / CHANGELOG.md / version.txt
```

Load-Bearing Files

These are the files where a single bug breaks an outsized portion of the system. Read these first when triaging:

Table 4.

Path	Role	Why it's load-bearing
packages/core/src/governance/decision.ts 51-60	Decision engine	The four-line if/else that produces every ship/warn/block/obsolete__review verdict. Every consumer reads its return value.
packages/core/src/schemas/evZod schema bundle/cls #54-145	Statement schemas	The cross-tool contract. Every IEP gate emits this shape. Until ia-j-E02 migrates to @intentsolutions/core@0.1.0, this file is the local source of truth — and a divergence from the kernel is a silent bug.
packages/cli/src/commands/emThe signing command	The signing command	The single producer of signed in-toto Statements in this repo. Cosign-subprocess error handling, artifact-hash-equals-input-hash check, predicate-body-only flag — all live here.
packages/db/src/lifecycle.ts 7-24	status state machine	Enforces the only legal transitions. A bug here lets a run end in an invalid state and breaks every downstream query that filters on status='completed'.

Path	Role	Why it's load-bearing
<code>packages/db/src/schema.ts:6-30</code>	Buzzle table definitions	The shape of every persisted row. Schema drift between this file and the application code is the most common breakage cause.
<code>packages/cli/src/commands/evaluate.ts:75-260</code>	Full pipeline orchestration	The end-to-end critical-path code. A bug in the orchestration order (e.g., transitioning to completed before persisting judgments) leaves the DB in an inconsistent state.
<code>packages/core/src/providers/TypescriptInfer.ts:150-187</code>	Proxies in contract	The contract every provider adapter must satisfy. The PB-7 measurement protocol is anchored to this surface.
<code>packages/cli/src/providers/asthrproviders:1-83</code>	ASTHR providers	Currently used unconditionally. Replacing these is the post-PB-7 single biggest behavior change.
<code>.github/workflows/release.yml:75-83</code>	Test step in release pipeline	The <code> true</code> after the test invocation means a failing test does not block a release. This is the single most concerning CI bug.
<code>scripts/audit-harness + .audit-harness/</code>	Vendored harness	If this vanishes or the hash drifts, every pre-commit gate fails. Hash-pinned via <code>audit-harness verify</code> .

6. Getting Started

Prerequisites

Table 5.

Tool	Version	Install	Verify
Node.js	>=20 (CI uses 22)	<code>fnm install 22 && fnm use 22</code> (or <code>nvm</code> equivalent)	<code>node --version -> v22.x.x</code>
pnpm	10.8.1 (pinned by <code>packageManager</code>)	<code>corepack enable && corepack prepare pnpm@10.8.1 --activate</code>	<code>pnpm --version -> 10.8.1</code>
Git	recent	system package manager	<code>git --version</code>
Optional: cosign	recent	https://docs.sigstore.dev/cosign/installation/	
Optional: bash	4+	system shell	needed for <code>scripts/audit-harness</code> wrapper

Zero to Running

1. Clone and enter

```
git clone https://github.com/jeremylongshore/j-rig-binary-eval.git
cd j-rig-binary-eval
```

2. Install dependencies (frozen-lockfile in CI; locally use either)

```
pnpm install
# Expect: "Done in <30s." plus a postinstall compile for better-sqlite3.
```

3. Run the full gate locally (lint + typecheck + test)

```
pnpm run check
# Expect: all green; ~28 test files pass; vitest summary at the bottom.
```

4. Build all packages

```
pnpm run build
# Expect: tsup emits to packages/{core,cli,db}/dist/.
```

5. Smoke the CLI without an API key (stub providers; see Section 4 Decision 7!)

```
node packages/cli/dist/index.js --version
# Expect: prints "0.0.0" (the CLI is private + workspace-internal; package version
# is intentionally 0.0.0 even though the repo is at v1.0.0)
node packages/cli/dist/index.js --help
# Expect: usage for 7 subcommands (check, validate, eval, report, optimize, drift,
# emit-evidence)
```

6. Run package integrity against a real skill

```
# (point at any directory containing a SKILL.md)
node packages/cli/dist/index.js check /path/to/some/skill
# Expect: deterministic checks; exit 0 if no errors.

# 7. Emit an unsigned in-toto Statement v1 (direct mode)
node packages/cli/dist/index.js emit-evidence \
  --gate-id 'j-rig:server:MM-1' \
  --result PASS \
  --input-hash sha256:e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855 \
  --policy-hash sha256:e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855
# Expect: a single line of JSON containing _type, predicateType, predicate, subject.

# 8. Verify the audit-harness hash-pinned policy files are unmodified
scripts/audit-harness verify
# Expect: exit 0 with "verified" message; exit 2 = HARNESS_TAMPERED.
```

Where the Rollout-Gate Decision Lives

A first-week engineer should know, by the end of day one, that `pnpm run check` is the gate that must pass before any PR, and that `decideRollout()` in `packages/core/src/governance/scoring.ts:51-60` is the function that decides `ship/warn/block/obsolete_review` for evaluated skills. The CLI's `eval` command (`packages/cli/src/commands/eval.ts:75-260`) is the orchestration site — read that file end-to-end to see the seven layers in order. The Evidence Bundle predicate URI is `https://evals.intentsolutions.io/gate-result/v1` (`packages/core/src/schemas/evidence-bundle.ts:1`) and the file that builds the signed envelope is `packages/cli/src/commands/emit-evidence.ts`.

Common Setup Problems

Table 6.

Symptom	Cause	Fix
Error: Cannot find module '@j-rig/core' when running CLI	Did not build first	<code>pnpm run build</code> — workspace <code>dist/</code> outputs are not source; the bin shebang points at compiled JS
Error: better-sqlite3 was compiled against a different Node version	Native binding skew (e.g., Node version changed after install)	<code>pnpm rebuild better-sqlite3</code> or delete <code>node_modules</code> + <code>pnpm install</code>
command failed: cosign attest-blob ... from emit-evidence --sign	cosign not on PATH	Install per the Sigstore docs; verify <code>cosign</code> version works
Invalid eval spec: ... unknown key	The eval-spec.yaml uses fields that don't match the Zod schema	Diff against a fixture in <code>packages/core/fixtures/valid/</code>

Symptom	Cause	Fix
All <code>j-rig eval</code> runs return “Ship” verdict	Stub providers are in use (Decision 7 cost surface)	Until PB-7 lands: do not use <code>j-rig eval</code> output as ground truth. Use <code>j-rig check</code> (deterministic) for real signal
<code>pnpm install</code> re-builds <code>better-sqlite3</code> every time	The <code>onlyBuiltDependencies</code> list whitelists the native compile; this is expected, not a bug	Accept the postinstall; CI caches it
<code>pnpm run check</code> reports lint errors only in <code>000-docs/</code>	The flat-config ignores array (<code>eslint.config.mjs:16-23</code>) should exclude <code>000-docs/</code> ; verify the ignore list	Re-run <code>pnpm install</code> if config changed
<code>tsc --noEmit</code> finds errors in test files but not source	<code>tests/tsconfig.json</code> is a separate project; root <code>tsc --noEmit -p tests/tsconfig.json</code> runs first per <code>package.json:19</code>	Fix the test file or update its <code>tsconfig</code>

7. Operations

Command Map

Table 7.

Task	Command	Notes
Install deps	<code>pnpm install</code>	Frozen-lockfile in CI; mutable locally
Run locally (full CLI)	<code>node packages/cli/dist/index.js <subcommand></code>	Build first
Build	<code>pnpm run build</code>	Recursive over workspaces; emits each package’s <code>dist/</code>
Build single package	<code>pnpm --filter @j-rig/core run build</code>	Useful for tight loops
Lint	<code>pnpm run lint</code>	ESLint flat config; auto-fix with <code>pnpm run lint:fix</code>

Task	Command	Notes
Format	<code>pnpm run format</code> (or <code>format:check</code> for CI)	Prettier
Typecheck	<code>pnpm run typecheck</code>	<code>tsc --noEmit</code> on tests + recursive on packages
Test	<code>pnpm run test</code>	vitest run (one-shot)
Test watch	<code>pnpm run test:watch</code>	vitest in watch mode
Full gate	<code>pnpm run check</code>	lint + typecheck + test; the local pre-merge bar
Clean	<code>pnpm run clean</code>	Recursive <code>rm -rf dist</code> per package
Deploy production	N/A — this is a CLI artifact, not a hosted service	The “deploy” is <code>git tag vX.Y.Z</code> -> CI cuts GitHub Release
View logs	N/A — no service	Run-time logs are <code>stdout/stderr</code> of the CLI
Rollback	N/A	Older versions live as historical GitHub Releases; consumers pin via npm-equivalent install of the tagged source
Run audit harness verify	<code>scripts/audit-harness verify</code>	Verifies hash-pinned policy files; exit 2 = <code>HARNESS_TAMPERED</code>
Run audit harness init	<code>scripts/audit-harness init</code>	After <code>ENGINEER-REVIEWED</code> edits to policy files; rewrites the <code>.harness-hash</code> manifest
Run escape-scan	<code>scripts/audit-harness escape-scan --staged</code>	Pre-commit gate; detects escape attempts in staged diff
Emit unsigned evidence	<code>node packages/cli/dist/index.js emit-evidence --gate-id ... --result PASS ...</code>	Direct-mode flags; outputs JSON to stdout
Emit signed evidence	<code>node packages/cli/dist/index.js emit-evidence --sign --key cosign.key --artifact <path> ...</code>	Requires cosign on <code>PATH</code> ; <code>--artifact</code> is mandatory under <code>--sign</code>

Deployment

This repo does not “deploy” to an environment in the traditional sense. The release artifact is a GitHub Release. The pipeline is `.github/workflows/release.yml`.

Pre-flight checklist 1. `pnpm run check` is green locally on the merge candidate. 2. No uncommitted changes (`release.yml:64-68` verifies this). 3. `CHANGELOG.md` is current (the workflow auto-prepends an entry, but a manual review is wise for messaging). 4. If a manual bump is needed: trigger `workflow_dispatch` with `bump: major|minor|patch`. Otherwise the auto path detects from conventional-commit prefixes (`BREAKING CHANGE`, `feat:`, anything else -> `patch`).

Execution steps - Auto path: push to main. The `release.yml on.push.branches: [main]` trigger fires. - Manual path: GitHub UI -> Actions -> Release -> Run workflow -> select bump type and dry-run flag.

Verification - The workflow’s “Verify readiness” step (`release.yml:62-83`) checks for uncommitted changes and runs tests. WARNING: the test step at `release.yml:73-74` uses `|| true`, so test failures do NOT block the release. Fix: change to `pnpm run test` without `|| true`. This is recorded as a high-priority finding in Section 11. - After the workflow completes, `git tag` shows the new tag; `gh release view vX.Y.Z` shows the release page.

Rollback protocol - There is no automatic rollback. To revert: `git revert <release commit> + push`, then trigger a new release. - To delete a bad release: `gh release delete vX.Y.Z + git push origin :refs/tags/vX.Y.Z`. Be aware that any downstream consumer that pinned to the deleted tag will break.

Monitoring & Alerting

- **Dashboards:** not configured. There is no hosted surface.
- **SLIs/SLOs:** not defined. The closest analog is CI green/red on PRs, which is a binary signal not a service-level objective.
- **On-call:** not established. Maintainer is Jeremy Longshore; `SECURITY.md` routes vulnerability reports to `security@jeremylongshore.com` with 24-hour ack target.
- **OTel emission:** `j-rig emit-evidence` emits a single `agent.rollout.gate.evaluated` event to `stderr` when `AUDIT_HARNESS_OTEL=1` or `OTEL_EXPORTER_OTLP_ENDPOINT` is set (`emit-evidence.ts:148-163`). There is no OTel SDK integration — this is a manual `stderr` write for parity with the `audit-harness` shell tooling.

Incident Response

Table 8.

Severity	Definition	Response Time	Playbook
P0	A release is published with a broken <code>decideRollout()</code> that ships when it should block	24 hours per <code>SECURITY.md</code> ack target	Revert the commit, cut a patch release, post-mortem in <code>000-docs/</code>

Severity	Definition	Response Time	Playbook
P1	A release passes CI but breaks the eval pipeline for downstream consumers	7 days per SECURITY.md High severity	Yank the release, identify the failing change, fix, cut a patch release
P2	A non-critical schema field is added without backward-compatible shim	30 days per Medium severity	Add the compat shim in a follow-up release; document the break
P3	Docs drift or a minor lint regression	90 days per Low severity	Fix on next normal release cycle

8. Things That Will Bite You

Ordered by likelihood x impact. These are real sharp edges from reading the code, not generic best-practices.

8.1 Stub providers shipped in the production code path

- **Symptom:** `j-rig eval` returns “Ship” verdict on every skill regardless of actual quality. Pipeline runs to completion, evidence is persisted, scores are computed — and every score is the stub default.
- **Cause:** `packages/cli/src/providers/anthropic.ts:13-83` defines three stub classes (`StubTriggerProvider`, `StubExecutionProvider`, `StubJudgeProvider`) that print “[stub] Would call ...” messages and return canned positive results. `eval.ts:34-37` imports them and uses them unconditionally; there is no real provider to fall through to until the PB-7 measurement run completes.
- **Fix:** Until PB-7 lands, use `j-rig check` (deterministic; real signal) and do NOT use `j-rig eval` output as ground truth. After PB-7: delete the stubs, replace with the locked adapter, or rename them `NullProvider` and gate behind `--stub`.
- **Prevention:** Add a loud terminal warning when stubs are active: `console.error(chalk.yellow("WARNING stub providers active; verdicts are synthetic"))`. File this as a bead before the next release.

8.2 The release workflow does not actually gate on tests

- **Symptom:** A commit with a failing test reaches `main`, the release workflow runs, the tag is cut, the GitHub Release is published — and the failing test was silently swallowed.
- **Cause:** `.github/workflows/release.yml:73-83` runs `pnpm run test || true`. The `|| true` means a non-zero exit code is ignored. The `ci.yml` workflow does gate on tests (no `|| true`), but the release workflow runs independently of CI status.

- **Fix:** Remove `|| true` from line 74. The job should fail if tests fail. Optionally add `needs: [test, lint, typecheck]` from the `ci.yml` jobs.
- **Prevention:** Treat the release workflow as an extension of the gate, not a separate concern. The pre-flight check should require CI green on the merge commit.

8.3 Schema duplication with `@intentsolutions/core@0.1.0`

- **Symptom:** An Evidence Bundle row passes validation in this repo but fails downstream when a consumer using the kernel schema validates the same row (or vice versa). The two Zod schemas have silently diverged.
- **Cause:** `packages/core/src/schemas/evidence-bundle.ts:1-12` carries a Zod mirror of the `gate-result/v1` spec authored before `@intentsolutions/core@0.1.0` published. The kernel now ships the canonical version. Both repos own a copy of the truth. The file's own header says “this file MUST stay in lock-step with that schema. Any divergence is a bug in this file (the lab spec wins).”
- **Fix:** Land `iaj-E02` — `@j-rig/core` imports the kernel schema and re-exports it rather than mirroring. The kernel version is pinned via `package.json` dependency; the lock-file is the source of truth.
- **Prevention:** Until `iaj-E02` lands, every PR touching `evidence-bundle.ts` must include a conformance-test diff against the lab spec's example fixtures (the test in `evidence-bundle.test.ts` is good but doesn't fail if the lab spec moves and this repo stays still).

8.4 `pnpm` workspace + native binding interactions

- **Symptom:** `pnpm install` succeeds but `node packages/cli/dist/index.js eval` throws `Error: better-sqlite3 was compiled against a different Node.js version.`
- **Cause:** The `better-sqlite3` native binding is built during `postinstall` against the current Node version. If the Node version changes between `install` and `run` (e.g., `nvm use` to a different version, or a CI step that swaps Node), the binding is stale.
- **Fix:** `pnpm rebuild better-sqlite3` or `rm -rf node_modules && pnpm install`.
- **Prevention:** `.nvmrc` pins 22. CI uses `cache: pnpm` to keep the binding fresh against the locked Node 22 image. Locally: always run `pnpm rebuild` after changing Node versions.

8.5 The `cosign` subprocess does not verify `cosign`'s version

- **Symptom:** A signed Statement is produced with an old `cosign` version that uses a deprecated DSSE shape; downstream verifiers reject it.
- **Cause:** `packages/cli/src/commands/emit-evidence.ts:280-295` invokes `cosign attest-blob` via `spawnSync` with no version check beforehand. If the user has `cosign v1.x` installed, the produced envelope may differ from what `v2.x` produces.
- **Fix:** Add a `cosign` version probe at the top of `signAndEmit()` and refuse versions below a known-good floor (e.g., 2.0.0). Document the supported range in README.
- **Prevention:** Pin a minimum `cosign` version in CI; document the install step in the README under “Optional: signing”.

8.6 The `--artifact sha256` mismatch failure has a friendly-but-misleading suggestion

- **Symptom:** A user runs `emit-evidence --sign --artifact ./out.json --input-hash sha256:abc...` and gets “sha256 mismatch” even though `out.json` is the file they care about.

- **Cause:** `emit-evidence.ts:243-249` correctly compares the artifact's hash to `predicate.input_hash`. But `input_hash` semantics are “the hash of the input the gate evaluated” — which may be a transformed version of `out.json` (e.g., the gate ran on the canonicalized JSON form, not the raw bytes). The error message says “the artifact passed to `--sign` must be the exact file whose hash the gate recorded” but does not tell the user how to discover what the gate recorded.
- **Fix:** When the mismatch is detected, also print the gate-id and reference to the source gate's docs (e.g., “this gate canonicalizes input via JCS before hashing; see docs/gate-id.md”).
- **Prevention:** Standardize input canonicalization across gates; document the canonicalization step in the Evidence Bundle SPEC.

8.7 The SQLite `--db` flag silently creates new files

- **Symptom:** A user runs `j-rig eval --db ./prod-runs.db` against a typo'd path; the harness creates a new empty database at the typo'd path and proceeds. Real evidence ends up in a stranded file the user can't find.
- **Cause:** `better-sqlite3` opens-or-creates by default. `lib/db.ts` does not require the file to pre-exist.
- **Fix:** Add a `--require-existing-db` opt-in flag, OR default to requiring the file to exist and add `--create-db` to permit creation.
- **Prevention:** For production CI runs, pre-create the DB file and use a CI script that fails if the file is unexpectedly missing.

8.8 The drizzle schema and the application code can drift without breaking tests

- **Symptom:** A new column is added to `runs` table in `schema.ts` but the application code doesn't read it. Or vice versa: the app code references a column that isn't in the schema. Tests pass because no test exercises the specific column.
- **Cause:** Drizzle's typed query builder catches *some* drift at compile time (referencing a non-existent column fails `tsc`), but cases like “schema has a column the app doesn't insert into” pass `tsc` silently — the column gets the default value or NULL.
- **Fix:** Per-table integration tests that round-trip every column through `insert + select`. The existing `evidence.test.ts` covers most of this but is not strictly exhaustive.
- **Prevention:** When changing `schema.ts`, also update at least one test that round-trips the new column. Add a property-based test that asserts `Object.keys(insertedRow)` `Object.keys(selectedRow)`.

8.9 The Evidence Bundle SPEC R9 cross-field invariant can be missed

- **Symptom:** A Statement passes the field-level Zod schemas but fails downstream verification because `subject[0].name !== predicate.gate_id` or `subject[0].digest.sha256` (no prefix) does not equal `predicate.input_hash` (with `sha256:` prefix).
- **Cause:** `evidence-bundle.ts:113-129` adds a `.superRefine()` that enforces this — but only when the *full* Statement is constructed. If a caller builds the predicate body separately and the subject separately and ships them in parallel (the `--predicate-body-only` mode in `emit-evidence`), the cross-field check is never run. Cosign then wraps the body in its own Statement v0.1 envelope with its own subject, and the produced envelope can be inconsistent.
- **Fix:** When `--predicate-body-only` is set, document loudly that the caller is responsible for matching subject to predicate. Better: do not allow `--predicate-body-only` together with `--sign` without an `--i-know-what-im-doing` flag.
- **Prevention:** Add a verification subcommand `j-rig verify-evidence <bundle-path>` that re-runs the cross-field checks against any envelope.

8.10 The audit-harness vendoring can drift from upstream

- **Symptom:** Upstream @intentsolutions/audit-harness releases a new version with a new gate; this repo’s vendored .audit-harness/v0.1.0 does not have it; pre-commit gates do not enforce the new check.
- **Cause:** The harness is vendored (scripts/audit-harness shells out to .audit-harness/scripts/*.sh). Vendored copies don’t auto-update.
- **Fix:** Run /sync-testing-harness periodically; or set AUDIT_HARNESS_VERSION=<latest> curl ... | bash from the install script.
- **Prevention:** Add the harness drift check to a monthly sweep. The parent 000-projects/CLAUDE.md already encodes this SOP.

9. Security & Access

Access Control

Table 9.

Role	Purpose	Permissions	MFA
Maintainer (Jeremy Longshore)	Repo owner; merges to main; cuts releases	Full GitHub admin on the repo	Yes (GitHub-enforced for Intent Solutions org)
Contributors	PR-only access	Open PRs against feature branches; cannot push to main	Yes (GitHub-enforced)
Bots (Dependabot, Gemini code-assist)	Automated dep PRs + AI code review	Read+PR scope; cannot merge	N/A
End users	Run the CLI on their machine	Local execution; no remote auth	N/A
security@jeremylongshore.com	Vulnerability intake mailbox	Per SECURITY.md	N/A

Secrets

- **In repo:** none. There is no .env.example because no runtime secrets are required by the CLI today (stub providers don’t call APIs). The Dependabot manifest does not require secrets.
- **In CI:** only secrets.GITHUB_TOKEN (auto-provided by GitHub Actions). The release workflow uses it for tag push + release create. No third-party PATs.
- **Rotation policy:** none documented. The repo is too young to have an established rotation cadence.

- **Emergency access:** Jeremy holds the `main` branch and release-cut authority. There is no documented break-glass procedure for the case where Jeremy is unreachable.

Honest Security Assessment

Implemented and verifiable: - Apache 2.0 license + NOTICE attribution (legal hygiene). - Pinned dependencies via `pnpm-lock.yaml` (supply-chain pinning). - Dependabot enabled (auto-PR for dep updates). - `pnpm run lint` and `pnpm run typecheck` gate every PR. - Hash-pinned `audit-harness` policy files; `scripts/audit-harness verify` exits 2 on tamper. - Cosign-signed in-toto attestations via `emit-evidence --sign` with Rekor transparency-log push (subprocess-driven, but functional). - CISO gate G-1 (credential redaction) is implemented and unit-tested at `packages/core/src/providers/ciso-gates/g1-credential-redaction.ts`. The harness wraps a provider invocation, captures stdout+stderr, and greps for 8+-char substrings of a synthetic test key. Zero matches = PASS. - CISO gate G-2 (env-var spillover) is implemented at `g2-env-var-spillover.ts`.

Aspirational or partial: - DNSSEC + CAA-record pinning on `evals.intentsolutions.io` is a prerequisite for first signed attestation per ISED CISO binding. NOT YET enforced at the DNS layer. - No SBOM is published with releases. The release workflow does not generate `cyclonedx` or `spdx` output. (Easy to add via `cyclonedx-bom` npm.) - The release workflow's test step is gated by `|| true` (see Section 8.2). A failing test does not block a release. This is a CI-integrity finding, not a runtime-security one, but it weakens the trust chain. - No security scanning beyond Dependabot. No CodeQL, no Snyk, no Socket. CodeQL is free for public repos and would catch a useful subset of bugs. - The `@intentsolutions/audit-harness` is vendored, not installed via npm. Vendoring is intentional (avoids tying every Intent Solutions repo to a single npm publish cadence) but means upstream security fixes don't propagate automatically.

Threat model gaps: - Arbitrary YAML loading. `parseAndValidateYaml` (`packages/core/src/parsers/yaml-p`) reads user-supplied eval specs. Zod validates the parsed object, but the `yaml` package itself could be vulnerable to malicious input that exploits a parser bug. The package is `yaml@2.8.3` which is recent and well-audited; the risk is low but non-zero. - Cosign subprocess invocation reads `--key` paths from CLI flags. A user who can supply CLI args could point `--key` at a sensitive file; cosign opens it. This is a local-only attack surface (the CLI is local), but worth noting. - The CLI does not sanitize skill paths. A `SKILL.md` at `/etc/passwd/SKILL.md` would fail the parser, but a `SKILL.md` inside a malicious skill directory could include arbitrary content the harness will print to stdout (chalk + console.log). No XSS surface (CLI output, not HTML), but log-injection is theoretically possible.

10. Cost & Performance

Monthly Costs

Table 10.

Resource	Cost	Notes
GitHub repo + Actions	\$0 (free tier for public repo)	CI minutes are unlimited for public repos
npm distribution	\$0 — none of the four packages are published	Internal monorepo only
GitHub Sponsors / Buy Me a Coffee	Variable inbound; not an outbound cost	.github/FUNDING.yml is set up
LLM API spend (per j-rig eval run)	Currently \$0 because stubs are active	Post-PB-7: per-eval cost depends on locked provider + token usage
Sigstore / Rekor	\$0 (public good infrastructure)	Subject to fair-use limits
Local development	\$0 in marginal cost	All tooling is OSS

Performance

- **Latency** (local, stub providers):
 - j-rig check: < 100 ms typical for a small skill (packages/core/fixtures/valid/)
 - j-rig validate: < 100 ms (parser + Zod)
 - j-rig eval end-to-end with stubs: < 1 s for a small spec (most time is SQLite open + drizzle setup)
 - j-rig emit-evidence (unsigned): < 50 ms
 - j-rig emit-evidence --sign: dominated by cosign subprocess (~500 ms cold, ~150 ms warm)
- **Throughput**: not measured. The CLI is a one-shot, not a server.
- **Error budget**: not defined.
- Once real providers are wired post-PB-7:
 - LLM-judge latency dominates. Anthropic Claude Sonnet completion at ~1-3 s per criterion; an eval with 10 criteria across 3 models is 30-90 s typical.
 - p-limit will throttle concurrent provider calls. The Provider.batch() method gives implementations a place to batch under the SDK's own primitive when available.

Scaling Limits

- **SQLite write throughput**: ~50k inserts/sec on a modern SSD for the small rows this schema uses; not a bottleneck for one-skill-at-a-time eval workflows. Multi-runner parallel evaluation against the same --db file is bottlenecked by SQLite's single-writer lock.
- **Memory**: better-sqlite3 is synchronous; large criterion_results reads bring rows into memory. A run with 100k criterion rows would consume ~50 MiB; not a problem until ~10M rows.
- **Provider rate limits**: post-PB-7, the dominant scaling limit. Anthropic's rate limits are per-organization; Vercel AI SDK and LiteLLM both surface 429 errors through the ProviderError shape. The harness has no built-in retry — that's expected to live in the adapter implementation.

- **CI time:** full pnpm run check runs in ~2 minutes on GitHub Actions ubuntu-latest; the limit is `tsc --noEmit` recursion across four packages.

11. Current State

What's Working

- The full pipeline runs end-to-end without an API key (`pnpm run check && node packages/cli/dist/index.js eval ./fixtures/valid/some-skill`). Evidence persists to SQLite; LaunchReport is produced; `decideRollout()` is exercised. (`packages/cli/src/commands/eval.`
- v1.0.0 was cut on 2026-05-19 and is Apache 2.0 with the NOTICE file populated. Relicensing CHANGELOG entry is clean. (`CHANGELOG.md:3-13`)
- All 10 epics are documented as shipped per the README; AARs land in 000-docs/ for Epics 02-08. (`000-docs/011-OD-REPT-epic-02-aar.md` through `017-OD-REPT-epic-08-aar.md`)
- The Evidence Bundle Zod schemas implement the gate-result/v1 spec field-for-field with cross-field invariants (R8, R9). (`packages/core/src/schemas/evidence-bundle.ts:54-145`)
- Cosign signing integration is wired with artifact-hash verification before signing. (`packages/cli/src/commands/emit-evidence.ts:209-303`)
- CISO gates G-1 and G-2 are implemented and tested. (`packages/core/src/providers/ciso-gates/`)
- The PB-7 measurement protocol is committed BEFORE the prototypes — the discipline the protocol itself enumerates as the failure mode it exists to prevent. (`000-docs/018-AT-SPEC-pb7-adapter-measurement-protocol-2026-05-12.md`)
- MM-1 through MM-6 OTel-trace failure-mode checkers are implemented with fixtures. (`packages/core/src/intentional-mapping/`)
- `@intentsolutions/audit-harness@v0.1.0` is vendored with a working wrapper. (`scripts/audit-harness, .audit-harness/VERSION`)
- CI runs lint, build, typecheck, and test on Node 22 with frozen-lockfile install. (`.github/workflows/ci.yml`)
- Dependabot is opening dep-update PRs; 2026-05 saw 4 such bumps land cleanly (#22, #27, #28, #36, #38).

What Needs Attention

- **[HIGH] Stub providers are the production code path.** `eval.ts:34-37` imports `StubTriggerProvider`, `StubExecutionProvider`, `StubJudgeProvider` from `packages/cli/src/providers/anthropic.ts:13-83` unconditionally. Every `j-rig eval` invocation today returns a synthetic “ship” verdict. -> Impact: any consumer treating `j-rig eval` output as ground truth is acting on synthetic data. -> Fix: post-PB-7, replace stubs with the locked adapter OR gate stub use behind a `--stub` flag with a loud warning OR rename to `NullProvider` and refuse to use without explicit opt-in.
- **[HIGH] Release workflow's test step uses `|| true`.** `.github/workflows/release.yml:73-83`. A failing test does not block a release. -> Impact: the release artifact's trust chain is broken at the most consequential gate. -> Fix: remove `|| true`; add `needs: [test, lint, typecheck]` from `ci.yml` jobs.
- **[HIGH] Schema duplication with `@intentsolutions/core@0.1.0`.** `packages/core/src/schemas/evid` mirrors the gate-result/v1 spec; the kernel now ships the canonical version. -> Impact: silent drift between this repo and the kernel becomes a bug in *this* repo (lab spec wins). -> Fix: land `iaj-E02` migration; `@j-rig/core` imports + re-exports the kernel schema.

- **[MEDIUM] evals.intentsolutions.io DNS pinning not yet in place.** ISEDC CISO binding requires DNSSEC + CAA-record pinning before first signed attestation. -> Impact: an attacker who compromises the DNS could redirect predicate-type resolution. -> Fix: configure DNSSEC on the domain; add CAA records pinning Let's Encrypt + Sigstore's CA.
- **[MEDIUM] @j-rig/dashboard is a placeholder.** Epic 10 team product is documented but not implemented. -> Impact: no team-level reporting UI; consumers have to query SQLite directly or read CLI output. -> Fix: scope Epic 10 build or deprecate the placeholder package.
- **[MEDIUM] All workspace packages are version: 0.0.0.** While intentional (none are published), the version string in the CLI output ("0.0.0") is misleading for users who built from a v1.0.0 git tag. -> Fix: have the release workflow sync workspace package versions to the repo version; OR have the CLI read from `version.txt` (the universal fallback the release workflow already writes).
- **[MEDIUM] No SBOM in releases.** -> Impact: downstream supply-chain auditors cannot pin the exact transitive tree of a release. -> Fix: add a `cyclonedx-bom` generation step to `release.yml`.
- **[LOW] cosign version not probed before subprocess.** `emit-evidence.ts` does not check cosign version. -> Impact: old cosign produces a different envelope shape than expected. -> Fix: add a version probe + minimum-version refusal.
- **[LOW] The --db flag silently creates new SQLite files.** -> Impact: typo'd paths create stranded DB files. -> Fix: add `--require-existing-db` opt-in.
- **[LOW] No CodeQL or third-party security scanning.** Dependabot only. -> Fix: enable CodeQL (free for public repos); evaluate Snyk/Socket for transitive analysis.
- **[LOW] Root-level tests/smoke.test.ts is trivial.** Tests `1 + 1 === 2`. -> Impact: no smoke confidence for the repo as a whole, only per-package vitest runs. -> Fix: replace with a real smoke that exercises the built CLI.

Implementation Status

Table 11.

Component	Status	Evidence
@j-rig/core schemas	IMPLEMENTED	<code>packages/core/src/schemas/</code> (8 files); Zod + cross-field invariants
@j-rig/core parsers	IMPLEMENTED	<code>packages/core/src/parsers/</code> (yaml, skill-md, agents-md)
@j-rig/core deterministic checks	IMPLEMENTED	<code>packages/core/src/checks/</code> + fixtures
@j-rig/core trigger layer	IMPLEMENTED	<code>packages/core/src/trigger/</code> + tests
@j-rig/core execution layer	IMPLEMENTED	<code>packages/core/src/execution/</code> + tests

Component	Status	Evidence
@j-rig/core judgment layer	IMPLEMENTED	packages/core/src/judgment/ + calibration + tests
@j-rig/core governance	IMPLEMENTED	packages/core/src/governance/ — decideRollout(), regression, baseline
@j-rig/core optimizer	IMPLEMENTED	packages/core/src/optimizer/ + tests
@j-rig/core drift	IMPLEMENTED	packages/core/src/drift/ + tests
@j-rig/core evidence I/O	IMPLEMENTED	packages/core/src/evidence/ + tests
@j-rig/core intentional-mapping	IMPLEMENTED (MM-1..MM-6)	packages/core/src/intentional-mapping/
@j-rig/core Provider interface	IMPLEMENTED (types only)	packages/core/src/providers/types.ts
@j-rig/core CISO gates G-1, G-2	IMPLEMENTED	packages/core/src/providers/ciso-gates/ + tests
@j-rig/core EC-1..EC-5 eval cases	IMPLEMENTED	packages/core/src/providers/eval-cases/
@j-rig/core score-card scorer	IMPLEMENTED	packages/core/src/providers/score-card/ + DR-draft generator
@j-rig/cli 7 subcommands	IMPLEMENTED (with stub providers)	packages/cli/src/commands/
@j-rig/cli provider implementation	STUB ONLY	packages/cli/src/providers/anthropic.ts
@j-rig/db schema	IMPLEMENTED	packages/db/src/schema.ts
@j-rig/db state machine + evidence I/O	IMPLEMENTED	packages/db/src/lifecycle.ts, evidence.ts
@j-rig/dashboard	PLACEHOLDER	packages/dashboard/package.json (no src/, no scripts)
Sigstore signing	IMPLEMENTED	packages/cli/src/commands/emit-evidence
Audit-harness vendoring	IMPLEMENTED (v0.1.0)	.audit-harness/VERSION, scripts/audit-harness
CI lint + typecheck + test	IMPLEMENTED	.github/workflows/ci.yml
Release automation	IMPLEMENTED (with test-gate bug)	.github/workflows/release.yml
Eval packs (Epic 10)	PLACEHOLDER	eval-packs/README.md

Component	Status	Evidence
Cross-tool Evidence Bundle dep on kernel	NOT YET	tracked as iaj-E02

12. Roadmap

Week 1 — Stabilization

- Remove `|| true` from release workflow test step. (Measurable: `release.yml` diff; next release fails on test break.)
- Add a loud warning when stub providers are active. (Measurable: `stderr` line on `j-rig eval` runs; first-time users cannot miss it.)
- Probe cosign version in `emit-evidence --sign`. (Measurable: refusal exit code for cosign < 2.0.0.)
- Replace `tests/smoke.test.ts` with a real CLI smoke test. (Measurable: smoke runs the built `node packages/cli/dist/index.js --version` and asserts output.)

Month 1 — Foundation

- Land iaj-E02: migrate `@j-rig/core` to consume `@intentsolutions/core@0.1.0` for the `gate-result/v1` schema. Delete the local mirror. (Measurable: `packages/core/src/schemas/evidence-bund` becomes a thin re-export; the kernel package shows up in `package.json dependencies`.)
- Complete the PB-7 measurement run between LiteLLM and Vercel AI SDK. Produce the Decision Record fragment via `draftDecisionRecordFragment()`. Lock the choice with an `ONN-AT-DECR-provider-adapter-choice-<date>.md` doc. (Measurable: DR exists; one of the two prototypes ships as the real adapter in `packages/cli/src/providers/`.)
- Replace stub providers with the locked adapter. Delete or rename the stubs. (Measurable: `j-rig eval` against a real skill produces a non-synthetic verdict.)
- Configure DNSSEC + CAA records on `evals.intentsolutions.io`. (Measurable: `dig +dnssec` shows DS records; CAA records pin Sigstore + Let's Encrypt.)
- Add SBOM generation to the release workflow (cyclonedx-bom or syft). (Measurable: every release page has an attached SBOM file.)

Quarter 1 — Strategic

- Build `@j-rig/dashboard` as a thin Next.js app that reads the SQLite evidence store and renders score-card history, regression trends, baseline comparisons. (Measurable: a working deployment under the partner portal or a sub-route.)
- Add CodeQL + a third-party SCA (Snyk or Socket). (Measurable: workflow files committed; PR checks include security signal.)
- Sync workspace package versions to the repo version during release. (Measurable: `@j-rig/core/package.json version` matches the latest tag.)
- Build a `j-rig verify-evidence <bundle-path>` subcommand that re-runs cross-field invariants on any in-toto envelope, including ones produced with `--predicate-body-only`. (Measurable: command exists, has tests, is documented.)

- Establish a vulnerability-disclosure cadence (quarterly rotation review + SECURITY.md update). (Measurable: a 000-docs/0NN-OD-SOPS-* doc lands documenting the cadence.)

13. Quick Reference

URLs

Table 12.

Resource	URL
Repo (GitHub)	https://github.com/jeremylongshore/j-rig-binary-eval
Releases	https://github.com/jeremylongshore/j-rig-binary-eval/releases
CI status	https://github.com/jeremylongshore/j-rig-binary-eval/actions/workflows/ci.yml
Evidence Bundle SPEC (lab)	https://github.com/jeremylongshore/intent-eval-lab/blob/main/specs/evidence-bundle/v0.1.0-draft/SPEC.md
Kernel package (npm)	https://www.npmjs.com/package/@intentsolutions/core
Audit harness (npm)	https://www.npmjs.com/package/@intentsolutions/audit-harness
Audit harness (GitHub)	https://github.com/jeremylongshore/audit-harness
Predicate URI (NOT yet DNSSEC-pinned)	https://evals.intentsolutions.io/gate-result/v1
Security intake	mailto:security@jeremylongshore.com

First-Week Checklist

- ☐ Read this document end-to-end (000-docs/019-AA-AUDT-...md)
- ☐ Read the master build blueprint (000-docs/007-PP-PLAN-master-build-blueprint.md)
- ☐ Read the PB-7 protocol (000-docs/018-AT-SPEC-pb7-adapter-measurement-protocol-2026-05-12.md)
- ☐ Clone repo and run `pnpm install && pnpm run check` to green
- ☐ Read `packages/core/src/governance/scoring.ts` end-to-end (the rules-engine core, ~110 LOC)
- ☐ Read `packages/cli/src/commands/eval.ts` end-to-end (the orchestration site, ~260 LOC)
- ☐ Read `packages/core/src/schemas/evidence-bundle.ts` (the cross-tool contract, ~145 LOC)

- ☐ Read `packages/cli/src/commands/emit-evidence.ts` (the signed-attestation producer, ~430 LOC)
 - ☐ Skim the 10 epic AARs in `000-docs/ONN-OD-REPT-epic-*.md`
 - ☐ Run `scripts/audit-harness --help` and `scripts/audit-harness verify`
 - ☐ (When unblocked) run `j-rig check` against a real `SKILL.md` and read the deterministic-registry output
 - ☐ Subscribe to `gh repo set-default jeremylongshore/j-rig-binary-eval` and watch the release page
-

Appendices

A. Glossary

- **Binary criterion:** a yes-or-no evaluation question. No partial credit. Either the criterion passed or it did not. The non-negotiable design principle in `CLAUDE.md:48-58`.
- **Blocker:** a criterion whose failure cannot be averaged out. Any blocker failure `-> decideRollout()` returns `block`.
- **Cosign:** the Sigstore signing CLI; produces DSSE envelopes; can push to Rekor transparency logs.
- **DSSE:** Dead Simple Signing Envelope; a binary signing format that wraps a payload + signature(s). <https://github.com/secure-systems-lab/dsse>
- **Evidence Bundle:** the cross-tool unification thesis. Every IEP gate emits an in-toto Statement v1 carrying a `gate-result/v1` predicate. Multiple statements compose a bundle.
- **EC-N (eval-case):** one of the five PB-7 measurement-protocol test surfaces (EC-1 single completion, EC-2 streaming, EC-3 tool calling, EC-4 error categories, EC-5 batching).
- **gate-result/v1:** the predicate URI <https://evals.intentsolutions.io/gate-result/v1> carried inside an in-toto Statement v1 to identify the evaluation predicate.
- **Intent Eval Platform (IEP):** the five-repo umbrella under `~/000-projects/intent-eval-platform/` that converges via the Evidence Bundle.
- **in-toto Statement v1:** a versioned attestation schema; `_type:` <https://in-toto.io/Statement/v1>; carries one or more subjects and a typed predicate. <https://github.com/in-toto/attestation>
- **ISED:** Intent Solutions Executive Decision Council — the 7-seat adversarial decision body that adjudicates architectural decisions.
- **MM-N (intentional-mapping category):** failure-mode checker that walks an OTel trace and classifies whether a known failure shape (async race, shape drift, cooldown, etc.) has manifested. MM-1 through MM-6 implemented.
- **PB-7:** the provider-adapter measurement protocol committed BEFORE the prototypes were written; § 6 enumerates CISO gates G-1 and G-2.
- **Rekor:** Sigstore's transparency log; an append-only Merkle-tree log of signed attestations.
- **Sacred regression:** a regression on a criterion flagged `regression_critical: true`. Blocks release regardless of average improvement.
- **SKILL.md:** the markdown artifact (with YAML frontmatter) that defines a Claude skill per the AgentSkills.io / Anthropic spec.

B. Reference Links

- Master build blueprint: `000-docs/007-PP-PLAN-master-build-blueprint.md`
- Architecture: `000-docs/003-AT-ARCH-architecture.md`

- Technical spec: 000-docs/005-AT-SPEC-technical-spec.md
- Status: 000-docs/006-OD-STAT-status.md
- Eval spec + contract guide: 000-docs/010-AT-SPEC-eval-spec-and-contract-guide.md
- Audit-harness baseline: 000-docs/010-TQ-SOPS-audit-harness-baseline-2026-05-01.md
- Epic AARs: 000-docs/{009,011,012,013,014,015,016,017}-OD-REPT-epic-*-aar.md
- PB-7 protocol: 000-docs/018-AT-SPEC-pb7-adapter-measurement-protocol-2026-05-12.md
- Parent CLAUDE.md: ~/000-projects/intent-eval-platform/CLAUDE.md
- Anthropic Claude Code skills doc: <https://code.claude.com/docs/en/skills>
- AgentSkills.io spec: <https://agentskills.io/specification>

C. Troubleshooting Playbooks

Playbook: pnpm install fails on better-sqlite3 native build

1. Confirm Node version matches .nvmrc (22): `node --version`
2. Confirm system has a C++ toolchain: `which g++ || sudo apt install build-essential`
3. Confirm Python is available (node-gyp): `python3 --version`
4. Clear cache: `pnpm store prune && rm -rf node_modules pnpm-lock.yaml`
5. Re-install: `pnpm install`
6. If still failing on Alpine: switch to Debian-based image (glibc, not musl).

Playbook: j-rig emit-evidence --sign exits with cosign signing failed

1. Verify cosign on PATH: `cosign version` (expect `>= 2.0.0`).
2. Verify the key file is readable: `cat $KEY_FILE | head -1`.
3. Verify the artifact path exists AND its sha256 matches `--input-hash`:

```
sha256sum <artifact-path>
# Compare to --input-hash (without the sha256: prefix)
```
4. Try keyless mode if key mode fails: `--keyless --rekor-url https://rekor.sigstore.dev`.
5. Capture cosign stderr: re-run with `-v` (cosign verbose) by setting `--cosign-bin "/path/to/cosign -v"` workaround.

Playbook: j-rig eval produces unexpected “Ship” verdict on a known-broken skill

1. Confirm the stub providers are active (they ARE today — pre-PB-7).
2. Use `j-rig check` instead — it is deterministic and produces real signal.
3. If you need actual functional evaluation, wait for PB-7 to lock the adapter, OR (temporarily) replace the imports in `packages/cli/src/commands/eval.ts:34-37` with a hand-rolled adapter against the Anthropic SDK and rebuild locally.

Playbook: Schema-divergence error when validating an Evidence Bundle from a different IEP repo

1. Confirm both producer and consumer are on compatible versions of the gate-result/v1 schema.
2. If consumer is `@intentsolutions/core@0.1.0` and producer is this repo’s `@j-rig/core`, both schemas should be field-equivalent today, but the `iaj-E02` migration is the durable fix.
3. Run the conformance test in `packages/core/src/schemas/evidence-bundle.test.ts` against the bundle to identify the diverging field.
4. File a bead documenting the divergence; the lab spec wins per the file header.

D. Open Questions

1. **When does @j-rig/dashboard get built?** Epic 10 marks it as the team-product layer. No active timeline. Defer until PB-7 + iaj-E02 land.
2. **Should the workspace packages publish to npm independently?** Today they are `private: true` with `version: 0.0.0`. The argument for publishing: external consumers could depend on @j-rig/core directly without vendoring. The argument against: this would force the four packages onto independent release cycles. Defer until external demand surfaces.
3. **Does the multi-tenant story matter for j-rig?** The DB schema has no `tenant_id`. The CLI is local-only. If j-rig becomes a hosted service (it has no hosted ambition per current scope), the answer changes. Tracked as `bd_000-projects-k0fj`.
4. **Is the stub-provider design correct or is it tech debt?** It enabled the rest of the system to be built; it is also the system's single biggest footgun (Section 8.1). Post-PB-7 it should go away cleanly. Worth a follow-up audit after PB-7 closes.
5. **Should @j-rig/cli's version be synchronized with the repo's git-tag version?** Today they diverge ("0.0.0" on the CLI, "1.0.0" on the repo). The release workflow already writes `version.txt`; should the CLI read from it? Decision deferred to the iaj-E02 sprint.
6. **What is the relationship between j-rig emit-evidence and audit-harness emit-evidence.sh?** Both produce in-toto Statements with the same predicate URI. The shell version is shipped in the audit-harness package; the TS version is here. They are kept in feature-parity by intent. Should they consolidate to one canonical implementation in @intentsolutions/core? Probably yes, post-PB-7. Until then they are documented as parallel implementations.
7. **Is the OTel "manual stderr write" pattern (emit-evidence.ts:148-163) durable, or should it move to an OTel SDK integration?** The stderr pattern matches the audit-harness shell tool; the SDK integration would be more robust but adds a dep. Hold for now; revisit when the rest of the IEP standardizes on an OTel posture.